Раскрытие проблемы организации пересылок MPI

Майстренко Марат 323, СКИ ВМК МГУ

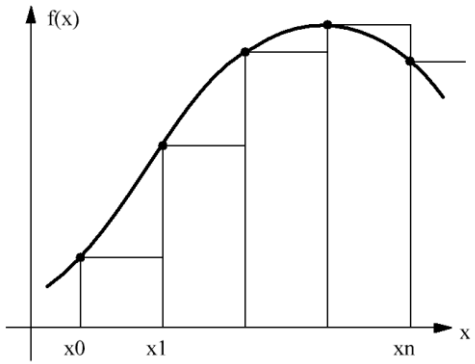
План доклада:

- Постановка проблемы
- Неправильный код
- Исправление кода
- Измерение ускорения. Анализ результатов
- Профилирование с помощью perf
- Профилирование с помощью mpiP
- Заключение

Постановка проблемы

- В современных вычислительных системах эффективная организация пересылок данных играет ключевую роль для достижения высокой производительности.
- Последовательная пересылка данных между процессами вместо одновременного обмена приводит к значительному снижению масштабируемости.
- Такие накладные расходы могут существенно ухудшить масштабируемость приложения, особенно при работе на большом количестве процессов.

Неправильный код



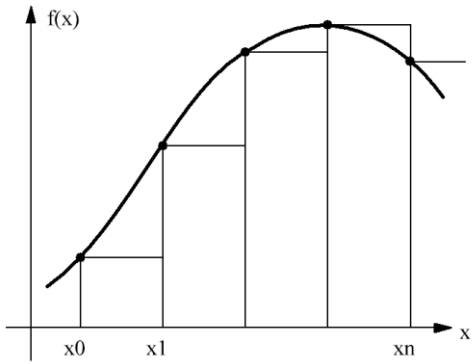
Метод левых прямоугольников

```
if (rank == 0) {
    double border_value = f(local_b) * h;
    MPI_Send(&border_value, 1, MPI_DOUBLE, rank + 1, 0, MPI_COMM_WORLD);
} else {
    double prev_border_value;
    MPI_Recv(&prev_border_value, 1, MPI_DOUBLE, rank - 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    local_sum += prev_border_value;

    double border_value = f(local_b) * h;
    if (rank < size - 1) MPI_Send(&border_value, 1, MPI_DOUBLE, rank + 1, 0, MPI_COMM_WORLD);
}

MPI_Allreduce(&local_sum, &total_sum, 1, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);

return total_sum;
```

- **<u>Основная проблема</u>** — последовательная пересылка данных с помощью MPI средств. В программе используются блокирующие операции MPI_Send и MPI_Recv.

- *В один момент* может происходить *только одна коммуникация*

Правильный код

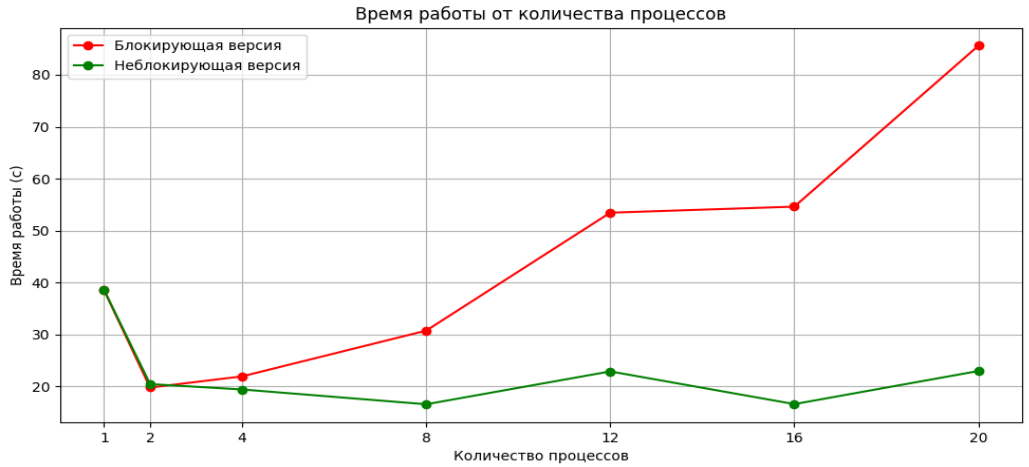


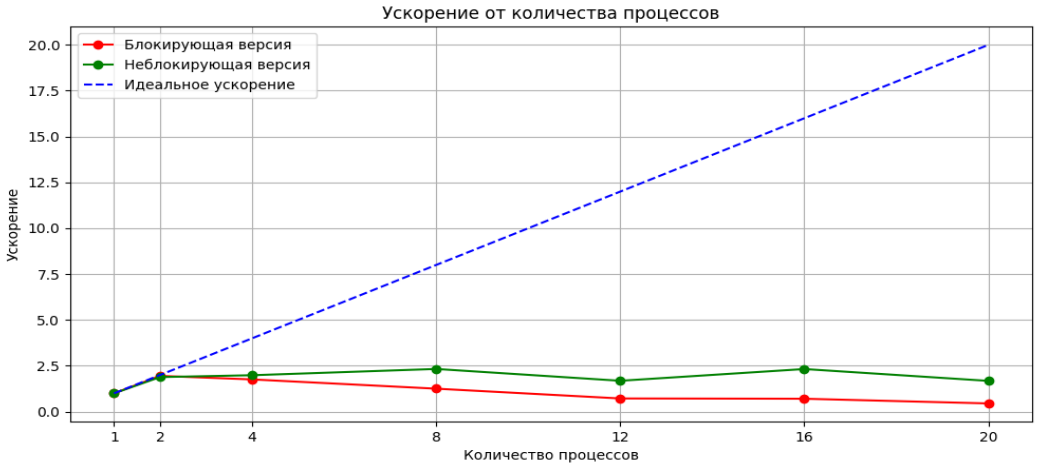
Метод левых прямоугольников

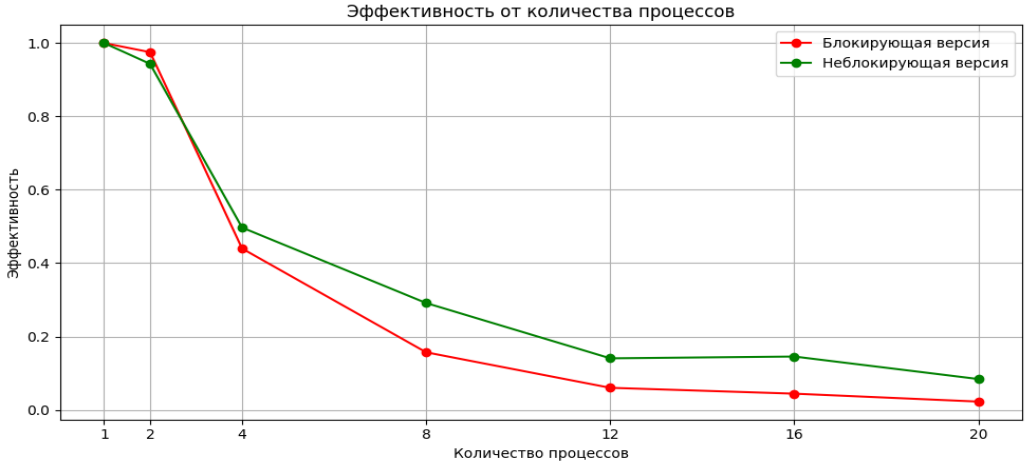
```
if (rank > 0) MPI_Irecv(&prev_border_value, 1, MPI_DOUBLE, rank - 1, 0, MPI_COMM_WORLD, &recv_request);

if (rank < size - 1) {
    border_value = f(local_b) * h;
    MPI_Isend(&border_value, 1, MPI_DOUBLE, rank + 1, 0, MPI_COMM_WORLD, &send_request);
}

if (rank > 0) {
    MPI_Wait(&recv_request, MPI_STATUS_IGNORE);
    local_sum += prev_border_value;
}

if (rank < size - 1) MPI_Wait(&send_request, MPI_STATUS_IGNORE);

MPI_Reduce(&local_sum, &total_sum, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

return total_sum;
```

- **Решение проблемы** — асинхронная и не избыточная отправка сообщений. Использование неблокирующих операций MPI_Isend, MPI_Irecv и MPI_Reduce вместо MPI_Allreduce

- Меньше *накладных расходы* и максимальная *эффективность передачи*







Измерение ускорения и анализ результатов

Неблокирующая версия (асинхронные операции):

Лучшая масштабируемость

Оптимальная производительность: Наибольшее ускорение достигается при **8 и 16 процессах** (~16.55–16.58 с)

Блокирующая версия (синхронные операции):

Плохая масштабируемость

Рост времени из-за накладных расходов

Профилирование с помощью perf

```
Samples: 25K of event 'cycles:ppp', Event count (approx.): 107355593312
  Children      Self  Command   Shared Object                    Symbol
+   97.83%     0.00%  zad_2     [unknown]                        [.] 0000000000000000
+   97.75%     0.00%  zad_2     libc-2.17.so                     [.] __libc_start_main
+   97.75%     0.00%  zad_2     libc-2.17.so                     [.] generic_start_main.isra.0
+   97.70%     0.16%  zad_2     zad_2                            [.] main
+   61.68%     0.79%  zad_2     zad_2                            [.] blocking_version
+   59.46%    12.02%  zad_2     libpami.so                       [.] PAMI_Context_advancev
+   52.49%     1.16%  zad_2     libcollectives.so                [.] LIBCOLL_Advance
+   51.87%     0.32%  zad_2     libmpi_ibm.so.2.0.1              [.] PMPI_Allreduce
+   51.25%     4.59%  zad_2     mca_coll_ibm.so                  [.] mca_coll_ibm_allreduce
+   34.85%     0.88%  zad_2     zad_2                            [.] non_blocking_version
+   26.26%     2.54%  zad_2     libpami.so                       [.] PAMI::Device::IBV::Device::advance
+   24.10%     0.26%  zad_2     libmpi_ibm.so.2.0.1              [.] PMPI_Reduce
+   23.78%     1.34%  zad_2     mca_coll_ibm.so                  [.] mca_coll_ibm_reduce
+   19.22%     1.29%  zad_2     libcollectives.so                [.] LibColl::NativeInterfaceP2P<true, false>::call_work_fn
+   13.40%    13.38%  zad_2     libpthread-2.17.so               [.] pthread_spin_lock
+   12.91%     0.00%  zad_2     libmlx5.so.1.0.0                 [.] 0xffffeffffe9db9a8
+   10.76%     1.88%  zad_2     libcollectives.so                [.] CCMI::Executor::HybridAllreduceT<CCMI::ConnectionManager::RankBasedConnMgr>::advance_fn
+   10.34%     0.13%  zad_2     libcollectives.so                [.] LIBCOLL_AutoSelect
+   10.06%     0.35%  zad_2     libcollectives.so                [.] LibColl::Adapter::StaticCollSelAdviser::autoSelect
+    6.95%     0.26%  zad_2     libopen-pal.so.2.1.0             [.] opal_progress
+    6.72%     0.41%  zad_2     libcollectives.so                [.] CCMI::Protocols::Allreduce::AsyncAllreduceFactoryT<CCMI::Protocols::Allreduce::HybridAsyncReduceT<CCMI::Conn
+    6.66%     6.65%  zad_2     libcollectives.so                [.] CCMI::Executor::ShmemShortBroadcastT<false, CCMI::Executor::ShmemAtomicBarrier>::advance_impl
+    5.42%     5.42%  zad_2     libm-2.17.so                     [.] __sin
+    5.34%     0.17%  zad_2     libmpi_ibm.so.2.0.1              [.] PMPI_Wait
+    5.17%     0.22%  zad_2     libmpi_ibm.so.2.0.1              [.] ompi_request_default_wait
+    5.13%     0.00%  zad_2     libmlx5.so.1.0.0                 [.] 0xffffeffffe9bb9a8
+    4.73%     0.04%  zad_2     libmpi_ibm.so.2.0.1              [.] PMPI_Recv
+    4.68%     0.13%  zad_2     mca_pml_pami.so                  [.] mca_pml_pami_recv
+    4.44%     0.40%  zad_2     zad_2                            [.] f
+    4.39%     3.81%  zad_2     libcollectives.so                [.] CCMI::Executor::ShmemBarrier::advance_impl
+    4.22%     0.02%  zad_2     mca_pml_pami.so                  [.] mca_pml_pami_progress
+    4.21%     0.12%  zad_2     mca_pml_pami.so                  [.] mca_pml_pami_progress_wait
+    3.63%     0.83%  zad_2     libcollectives.so                [.] CCMI::Executor::HybridReduceT<CCMI::ConnectionManager::CommSeqConnMgr>::advance_fn
+    3.13%     0.16%  zad_2     libpami.so                       [.] PAMI_Send_immediate
+    2.82%     2.82%  zad_2     libcollectives.so                [.] CCMI::Executor::ShmemReduceT<CCMI::Executor::ShmemAtomicBarrier>::advance_impl
+    2.56%     0.00%  zad_2     libmlx5.so.1.0.0                 [.] 0xffffeffffe9dbdc0
+    2.19%     0.08%  zad_2     libpami.so                       [.] PAMI::Protocol::Send::Eager<PAMI::Device::Shmem::PacketModel<PAMI::Device::ShmemDevice<PAMI::Fifo::LinearFif
+    2.04%     0.09%  zad_2     libcollectives.so                [.] CCMI::Executor::HybridReduceT<CCMI::ConnectionManager::CommSeqConnMgr>::reset
+    2.00%     0.07%  zad_2     libcollectives.so                [.] CCMI::Protocols::Allreduce::ProtocolFactoryT<CCMI::Protocols::Allreduce::HybridOATCompositeT<CCMI::Executor:
+    1.92%     0.06%  zad_2     libcollectives.so                [.] CCMI::Executor::Allreduce<CCMI::ConnectionManager::CommSeqConnMgr>::reset
```

Параметры perf для профилирования:

Частота сэмплирования – 1000 Гц (-f 1000)

Сбор цепочек вызовов (- g)

Профилирование с помощью mpiP

```
@--- MPI Time (seconds) ------------
-------------------------------------
Task    AppTime    MPITime     MPI%
   0       64.8       55.5    85.68
   1       64.8       52.7    81.37
   2       64.8       52.6    81.22
   3       64.8       52.6    81.24
   4       64.8       52.8    81.56
   5       64.8       52.6    81.19
   6       64.8       51.5    79.48
   7       64.8       52.7    81.43
   *        518        423    81.65
```

```
@--- Aggregate Time (top twenty, descending, milliseconds) ----------------
-----------------------------------------------------------------------------
Call                 Site       Time    App%    MPI%     Count    COV
Allreduce               3   1.79e+05   34.51   42.26  40000000   0.46
Recv                   10   1.06e+05   20.41   24.99  35000000   0.62
Reduce                 11   6.82e+04   13.16   16.12  35000000   0.09
Wait                    9   2.17e+04    4.18    5.12  35000000   0.02
Reduce                  4   1.54e+04    2.98    3.65   5000000   0.00
Isend                   7   1.01e+04    1.96    2.39  35000000   0.05
Send                    8   9.25e+03    1.78    2.19  30000000   0.04
Irecv                  12   7.94e+03    1.53    1.88  35000000   0.01
Wait                    2    4.3e+03    0.83    1.02  35000000   0.04
Send                    5   1.63e+03    0.31    0.39   5000000   0.00
Barrier                 1       3.23    0.00    0.00         8   0.84
Barrier                 6     0.0422    0.00    0.00         8   0.04
```

- **Основным узким местом** является операция **Allreduce**, занимающая **34.51%** общего времени.
- Point-to-point операции **Send/Isend/Recv/Irecv** суммарно занимают **25.68%** времени

Заключение

- Исследование наглядно демонстрирует **важность правильной организации обменов** с помощью MPI в параллельных вычислениях.

- **Накладные расходы** на синхронизацию обмена **могут значительно превышать выигрыш** от параллельного выполнения задач